

Keeping the app up to date

Mambanda Market

August 24, 2026

CONTENTS

The short version	
What is already in place	
What “auto-update” can actually mean	
The blocking question	
Do not block on “a newer build exists”	
Rules the gate must follow	
Recommended plan	
Phase 1 — Manifest check, now	
Phase 2 — Play in-app updates, at store launch	
Phase 3 — Over-the-air code push, only if the evidence changes	
Admin dashboard controls	
Risks	
What to do next	

The short version

Three things are worth deciding separately, and they are usually run together by mistake:

1. **Telling people a new build exists.** Cheap, safe, and not yet built.
2. **Refusing to let an old build keep running.** Powerful, occasionally necessary, and the fastest way to take your own userbase offline if it is wired to the wrong signal.
3. **Shipping a fix without a new build at all.** Attractive in principle; under Flutter it covers a narrower set of changes than it first appears.

The recommendation is to build (1) now on top of what already exists, design (2) carefully but leave it switched off until it is needed, and defer (3) until the release history says it would actually have helped — which today it does not.

What is already in place

Most of the machinery for this exists. It was built for other reasons and happens to be exactly the right foundation.

Piece	Where	Status
Update manifest — version, build, URL, size, SHA-256, min Android, notes	<code>latest.json</code> on the site	Live, already published on every release
Verified download endpoint	<code>/download</code> → current APK	Live
Admin-editable settings, no release required	<code>app_settings</code> → <code>GET /config</code>	Live, seeded by migration 0026
Client config with local cache that fails open	<code>RemoteConfig</code>	Live
Admin dashboard	<code>mambandamarket-admin</code>	Live

`latest.json` is already a complete update manifest. It carries the build number, the download URL and a SHA-256 that we verify on every release. What is missing is not the manifest — it is a client that reads it. The app currently has no idea what build it is, relative to what is available.

`RemoteConfig` matters just as much, and for a subtler reason: it reads its cache first, then refreshes from the network, and it never throws. That behaviour is the single most important property of anything that can block the app, and it is already the house pattern.

What “auto-update” can actually mean

The honest answer differs by how the app got onto the phone, and today that is the sideloaded APK.

	Sideloaded APK (today)	Play Store (planned)
Silent background update	Not possible	Yes, Play does it by default
App can detect a new version	Yes, by reading the manifest	Yes, via the in-app updates API
App can start the install	Only by handing the APK to the system installer, with <code>REQUEST_INSTALL_PACKAGES</code> ; the user still taps Install	Flexible or immediate flows, handled by Play
App can hard-block old versions	Only by our own gate	Only by our own gate
Time from release to reaching users	Immediate	Review, then staged rollout

So on the current channel, “auto-update” honestly means **automatic detection and a one-tap path to installing**, not automatic installation. That is worth being clear about internally, because it changes what the admin toggle is promising.

One thing does not change across channels: **neither Play nor the APK gives you a reliable hard block**. Play’s immediate update flow can be dismissed in practice, and sideloaded installs ignore Play entirely. If blocking is a real requirement, it has to be ours, which is the next section.

The blocking question

This is the part worth getting right, because it is the only item on the list that can break the product for everyone at once.

Do not block on “a newer build exists”

The instinct is one number: latest build, and anything older is stale. That couples two unrelated decisions and produces an app that nags — or worse, blocks — on every routine release, including the ones that change a label.

Use **two** numbers instead:

- `update.latest_build` — what is available. Drives a dismissible nudge.
- `update.min_supported_build` — below this, the build is no longer allowed to run. Drives the hard gate, and moves **rarely**.

Most releases move only the first. The second moves when an old client is genuinely unsafe or broken:

- the API contract changed underneath it,
- it displays money wrongly (a build that showed prices 100x off would be a real example, and that class of bug has already happened here once),
- a security fix,
- it writes data the current server can no longer accept.

Everything else is a nudge.

Rules the gate must follow

Fail open. If the config request fails, the user gets in. A gate that depends on our API being reachable converts any API blip into a total outage, on every phone, with no way to recover except fixing the API. `RemoteConfig` already behaves this way — the gate must not be the thing that changes it.

Reversible in seconds. Raising the floor is one row in `app_settings`, and so is lowering it. No release, no deploy. If a floor is set wrongly at 9pm it must be undone at 9:01, from the admin dashboard, by whoever is awake.

Off by default. `min_supported_build` defaults to `0` and `update.mode` defaults to `notify`. The dangerous setting should require someone to deliberately turn it on, the same reasoning already applied to `PAYMENTS_SANDBOX` in the API environment.

Grace before it bites. A soft warning for some days before the floor applies, so the blocking screen is never the first time anyone hears about it.

Never a dead end. The blocking screen must offer the download, a retry, and a plain sentence saying why. A screen that only says “update required” with no working button is indistinguishable from the app being broken.

Measure before raising it. We currently have no idea how many installs are on which build, because nothing reports it. Raising the floor without that is guesswork that can lock out most of the userbase. The cheapest fix is for `ApiClient` to send an `X-App-Build` header on every request and for the API to record it — a day’s work, and it turns the floor from a gamble into a decision. **This should land before the gate is ever switched on.**

Recommended plan

Phase 1 — Manifest check, now

The app reads the manifest on launch and on resume, compares build numbers, and does one of three things according to `update.mode`:

- `off` — nothing.

- `notify` — a dismissible prompt when `latest_build` is higher. Remembers the dismissal so it is not shown on every resume.
- `block` — a full-screen gate when the running build is below `min_supported_build`, subject to every rule above.

Alongside it, `X-App-Build` on outbound requests so version spread becomes visible. Estimated at two to three days including the admin controls, and it works on the current sideload channel with no store dependency.

Phase 2 — Play in-app updates, at store launch

Add the Play in-app updates flow for installs that came from Play, and keep the Phase 1 gate as the floor beneath it, because sideloaded installs will keep existing. Play handles the common case silently; our gate stays for the rare case Play cannot serve.

Two things to carry into that migration, both consequences of decisions already made:

- **Play App Signing re-signs the app**, so the certificate fingerprint changes. `.well-known/assetlinks.json` on the site currently carries the upload keystore's fingerprint. The Play signing fingerprint must be added there before the store build ships, or every shared listing link stops opening in the app and falls back to the browser.
- The manifest and the store listing become two sources of truth for “current version”. `latest.json` should keep describing the APK channel specifically.

Phase 3 — Over-the-air code push, only if the evidence changes

Shorebird is the credible option for Flutter. It replaces compiled Dart at launch, without a store round trip. It cannot change native code, plugins, dependencies, or the Android manifest.

That last constraint is the whole argument, and our own recent history answers it. Of the last four substantive changes:

Change	Would OTA have shipped it?	Why
Unread badge not clearing	Yes	Dart only
Sign-in navigation fix	Partly	The guard is Dart; it landed with no manifest change, but the splash ordering did not need one either
Push notifications	No	New plugins, manifest entries, Gradle changes
Listing sharing	No	New plugin, new intent filter, <code>assetlinks.json</code>

Roughly one in four. OTA would not have carried the work that actually mattered this month, and it adds a vendor, a cost, and a second artifact to reason about when something goes wrong. Revisit when the release cadence is high **and** the changes have become mostly Dart — which is what a mature app looks like, and this one is not there yet.

Admin dashboard controls

New rows in `app_settings`, exposed through `/config` and edited in the admin panel like the sign-up settings already are.

Key	Type	Default	Effect
<code>update.mode</code>	<code>off</code> <code>notify</code> <code>block</code>	<code>notify</code>	Master switch. <code>off</code> is the kill switch.
<code>update.latest_build</code>	int	current	Drives the dismissible nudge.
<code>update.latest_version</code>	string	current	Shown in the prompt.
<code>update.min_supported_build</code>	int	<code>0</code>	The hard floor. <code>0</code> never blocks.
<code>update.blocks_from</code>	ISO date, nullable	null	Floor applies only from this date; warn before.
<code>update.message_en</code> / <code>update.message_fr</code>	string	null	Optional line explaining why, in both shipped languages.

Reusing `app_settings` rather than inventing a mechanism means these inherit the caching, the fail-open behaviour and the admin UI that already work.

Risks

Risk	Containment
A wrong floor locks out the userbase	Default <code>0</code> ; <code>mode: off</code> kill switch; reversible from admin in seconds
Config unreachable, everyone blocked	Fail open — no answer means let them in
Users nagged on every routine release	Nudge and gate driven by separate keys; dismissal remembered
Floor raised blind, most users locked out	<code>X-App-Build</code> telemetry lands first, and is a precondition
Shared links break after Play migration	Add the Play signing fingerprint to <code>assetlinks.json</code> before the store build
Sideloaded users cannot install the update	Manifest already carries the SHA-256; verify the download before handing it to the installer

What to do next

1. `X-App-Build` header, and record it server-side. Nothing else should start before version spread is visible.
2. Phase 1 client check with `mode: notify`. Ship it; leave `block` alone.
3. Add the six `app_settings` rows and the admin controls.
4. Only once (1) shows the spread, consider setting a floor — and only for a release that genuinely warrants it.
5. Revisit Play in-app updates when the store listing is real, carrying the `assetlinks.json` fingerprint change with it.